

COM207-Data Structure

Dictionarys & Hashing

Dr. Salwa Osama

Dictionary

- A dictionary is composed of a collection of elements each of which is:
 - Set of (key, value) pairs
 - keys are mapped to values
 - keys must be comparable
 - keys must be unique
- **Examples:**
 - Membership in a club
 - Course you have passed
 - Set of loans made in a library
 - Language dictionary

Search key	Data item
------------	-----------

A dictionary entry

Dictionary Example

Dictionary

Course Record



Member
Record

key	student name	hw1	
123	Stan Smith	49	...
124	Sue Margolin	56	...
125	Billie King	34	...
⋮			
167	Roy Miller	39	...

Dictionary Example

To count the occurrences of words in a story:

- Each dictionary entry is keyed by the word
- The related value is the count
- When entering words into dictionary
 - Check if word is already there
 - If no, enter it with a value of 1
 - If yes, increment its value

Dictionary Operations

- Test whether dictionary **is empty**.
- Test whether dictionary **is full**.
- **Insert** new item into dictionary.
- **Remove** item with given search key from dictionary.
- **Remove all items** from dictionary.
- **Get item with a given search key** from dictionary.
- **Test whether dictionary contains an item** with given search key.

Possible Implementations of Dictionary?

- Trees (BST, AVL, Splay,....)
- Lists
- Hash tables

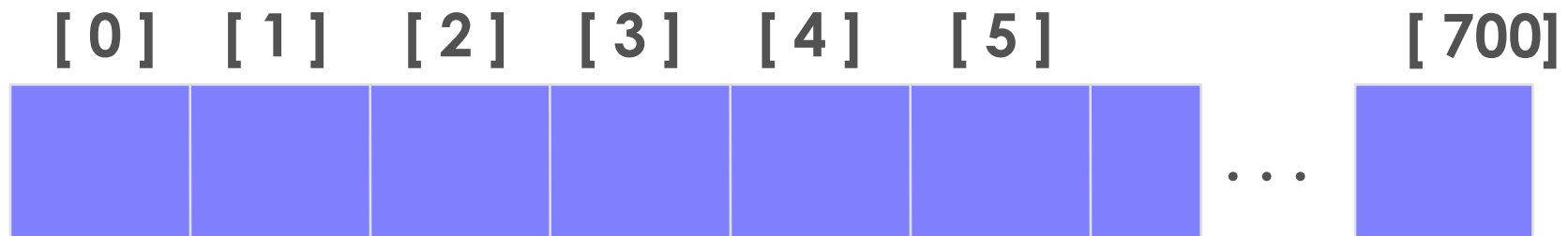
Hash Tables

Hash table

- A Hash table is defined as a data structure used to insert, look up, and remove key-value pairs quickly. It operates on the hashing concept, where each key is translated by a hash function into a distinct index in an array. The index functions as a storage location for the matching value. In simple words, it maps the keys with the value.

What is a Hash Table ?

- Effective way of implementing dictionaries.
- The simplest kind of hash table is an array of records.
 - *Element whose key is k is obtained by indexing into the k^{th} position of the array.*
- Applicable when we can afford to allocate an array with one position for every possible key.



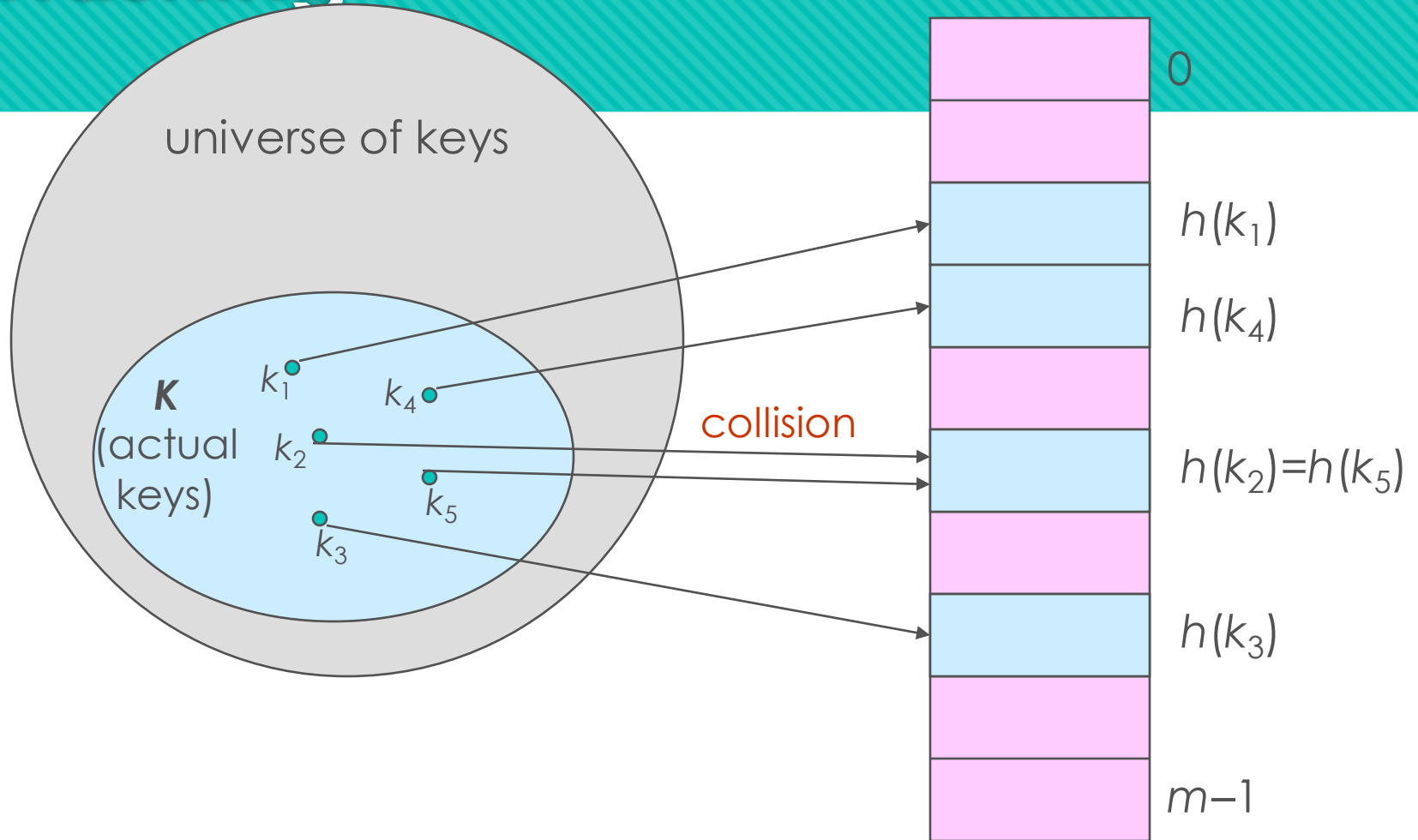
Hashing

- Use *hash function* to map keys into positions in a hash table

Ideally:

- If element e has key k and h is hash function, then e is stored in position $h(k)$ of table
- To search for e , compute $h(k)$ to locate position. If no element, dictionary does not contain e .

Hashing



Hash Table

- The ADT **Hash Table** is an array of elements (associated with a search key unique for each element), together with an **hash function** and **access procedures** (insert, delete, search...).
- The **hash function** determines the location in the table of a new element, using its search key. In a similar way, permits to locate the position of an existing element.
- The **hash function** takes a search key and maps it into an integer array index.

Properties of Good Hash Functions

- Must return number 0, ..., tablesize
- Should be efficiently computable.
- Should minimize collisions
= different keys hashing to same index

Truncation in Hashing

Truncation in Hashing:

- **Definition:**

Truncation is a simple method for creating a **hash function** by **ignoring part of the key** and using certain selected digits (or parts) from the key to generate an index.

Truncation in Hashing Example

Let's say you have:

- **Keys:** 8-digit numbers (e.g., 21296876)
- **Hash Table Size:** 1000 locations (i.e., indices from 0 to 999)

You want to generate a hash value for the key 21296876 .

Using truncation, you might pick:

- 1st digit from the right: 6
- 2nd digit from the right: 7
- 5th digit from the right: 9

So you form: 976

→ The key 21296876 is mapped to index 976.

Problem with Truncation:

While truncation is **very fast**, it's **not always effective** because:

- It may not **spread keys evenly** across the table.
- This could cause **collision** (many keys going to the same index), which reduces performance.

Folding in Hashing

- **Folding** is a hashing technique where the **key is split into parts**, and those parts are **combined** (usually by **adding** or **multiplying**) to produce a hash value (index).

Folding in Hashing Example

Suppose the key is:

21296876 (8-digit number)

You split it into parts:

- First 3 digits: 212
- Next 3 digits: 968
- Last 2 digits: 76

Add them together:

$$212 + 968 + 76 = 1256$$

If your hash table only has 1000 positions, take only the last 3 digits (i.e., **truncate**):

→ 1256 becomes 256

So, 21296876 maps to index 256.

Modular Arithmetic in Hashing

This hashing method uses **modulus division** to calculate the index.
The hash function is:

$$h(\text{key}) = \text{key} \bmod \text{TableSize}$$

That means:

- Divide the key by the size of the table.
- Take the **remainder** as the index.

Modular Arithmetic in Hashing Example

Let's say:

- Key = 21296876
- Table Size = 1000

Then:

$$h(21296876) = 21296876 \bmod 1000 = 876$$

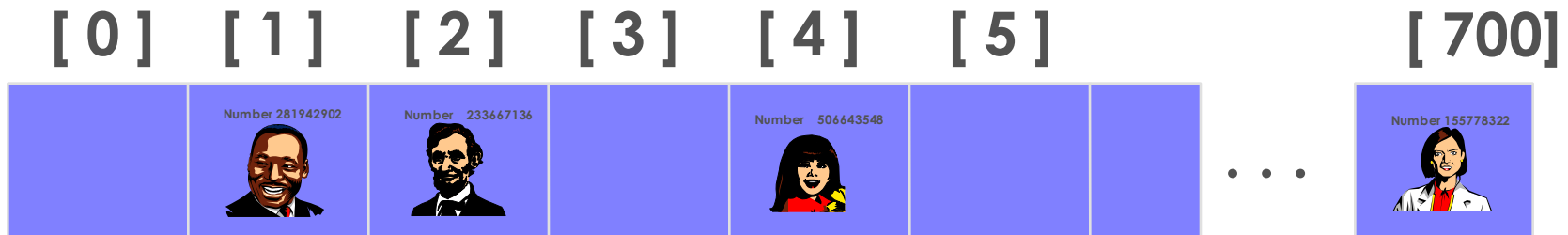
So the key 21296876 is stored at index 876 in the table.

Let's Try...

Inserting a New Record

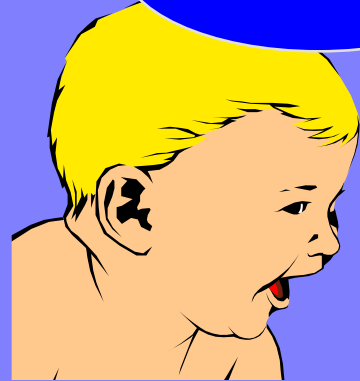


- In order to insert a new record, the key must somehow be converted to an array index.
- The index is called the hash value of the key.



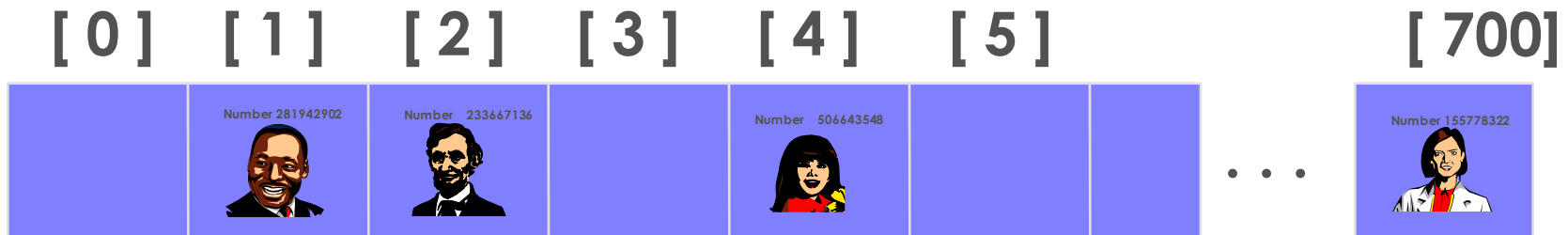
Inserting a New Record

Number 580625685



What is $(580625685 \bmod 701)$?

- Typical way create a hash value:
(**Key mod TableSize**)

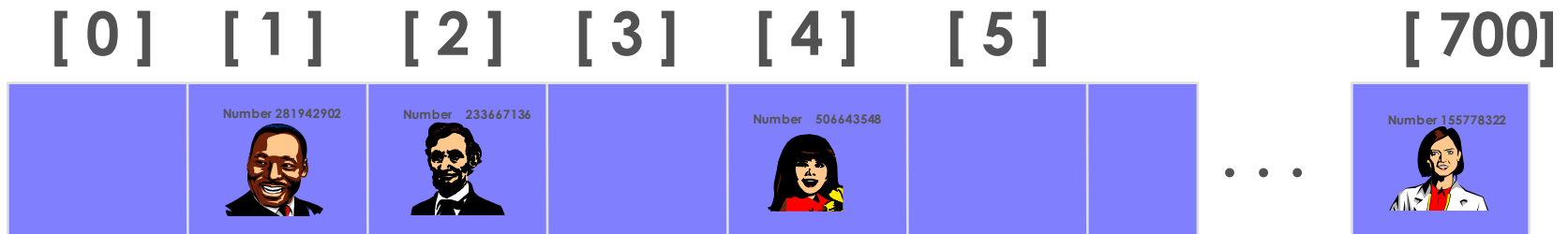


Inserting a New Record

(Number mod 701)

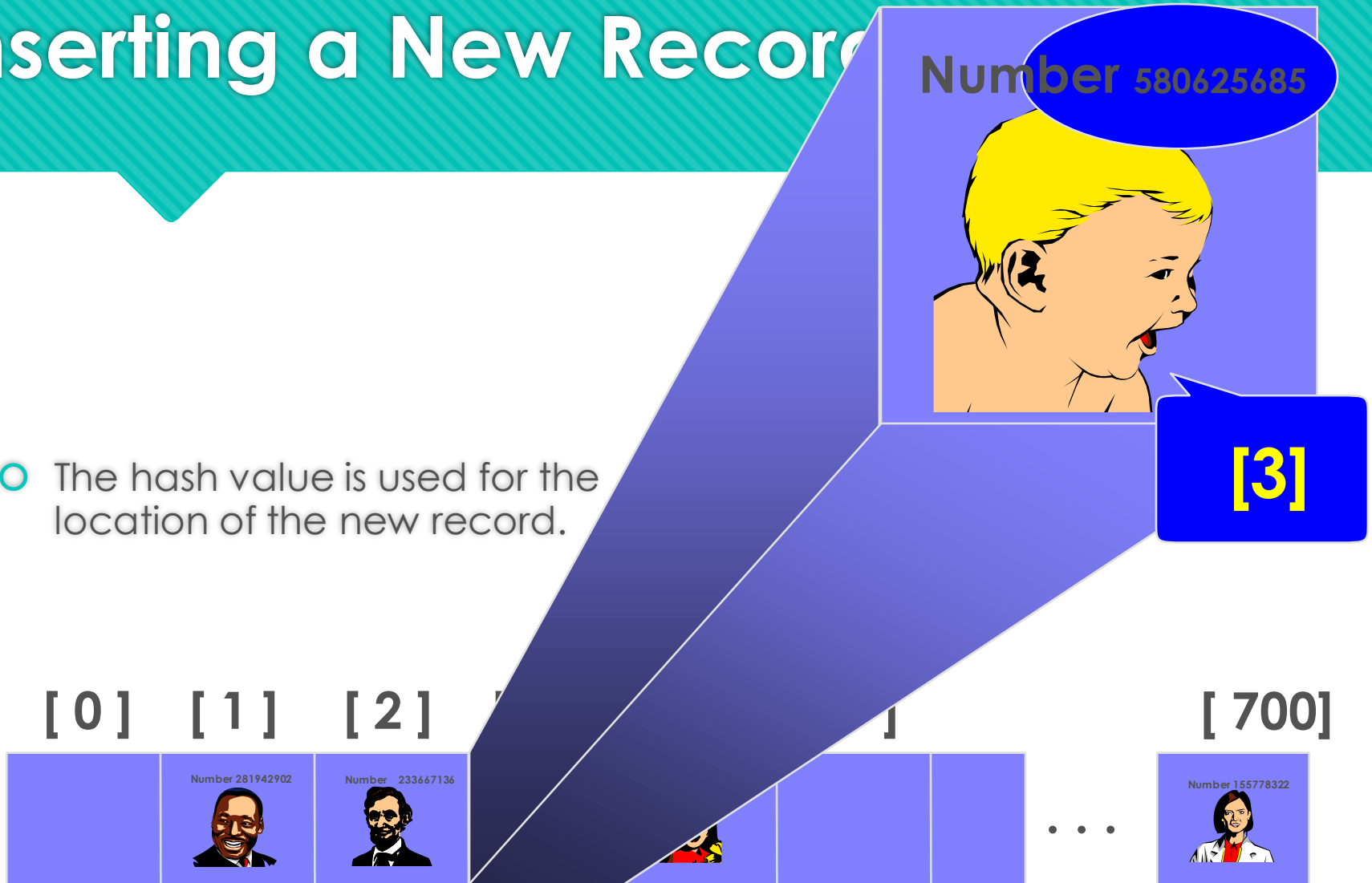
What is $(580625685 \bmod 701)$?

- Typical way to create a hash value:



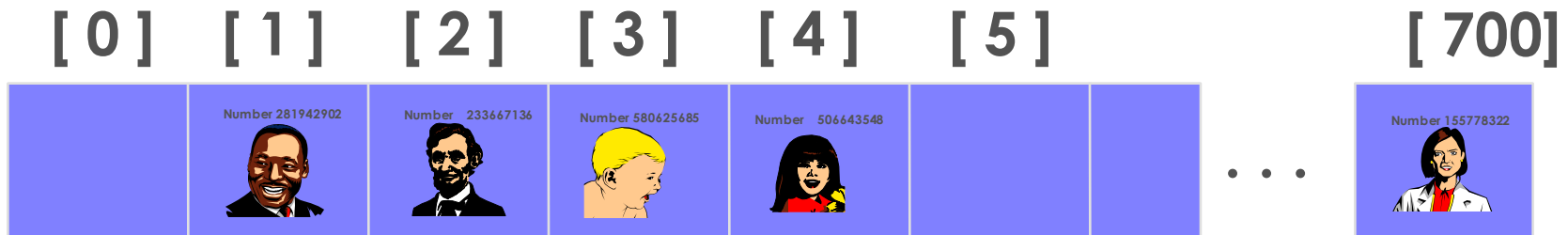
Inserting a New Record

- The hash value is used for the location of the new record.



Inserting a New Record

- The hash value is used for the location of the new record.

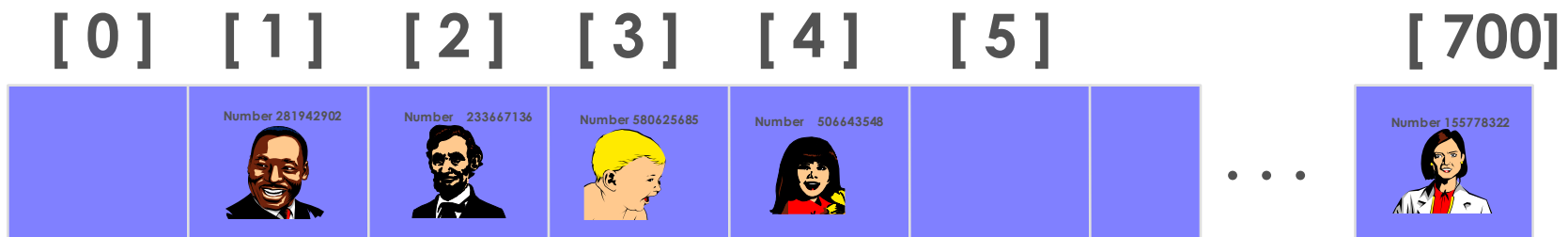


Collisions

- Here is another new record to insert, with a hash value of 2.



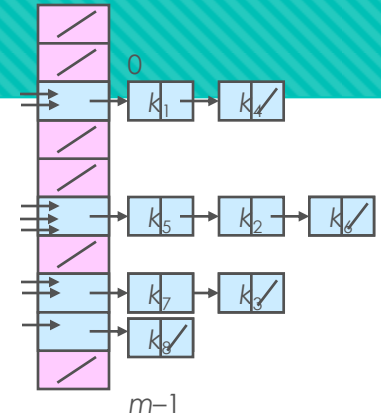
My hash value is [2].



Methods of Collision Resolution

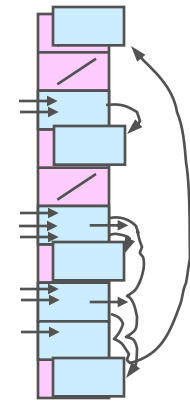
○ Chaining:

- Store all elements that hash to the same slot in a linked list.
- Store a pointer to the head of the linked list in the hash table slot.

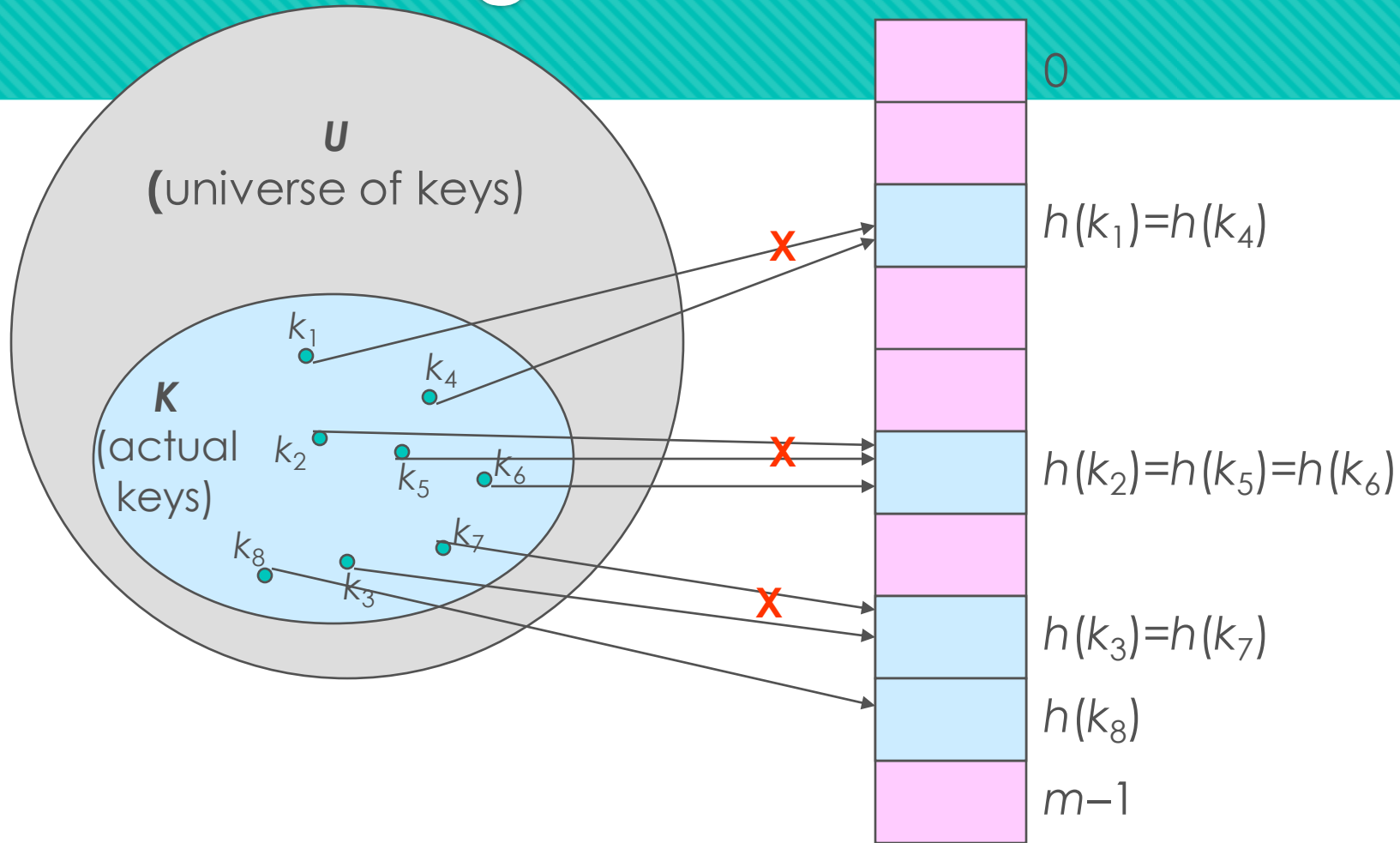


○ Open Addressing:

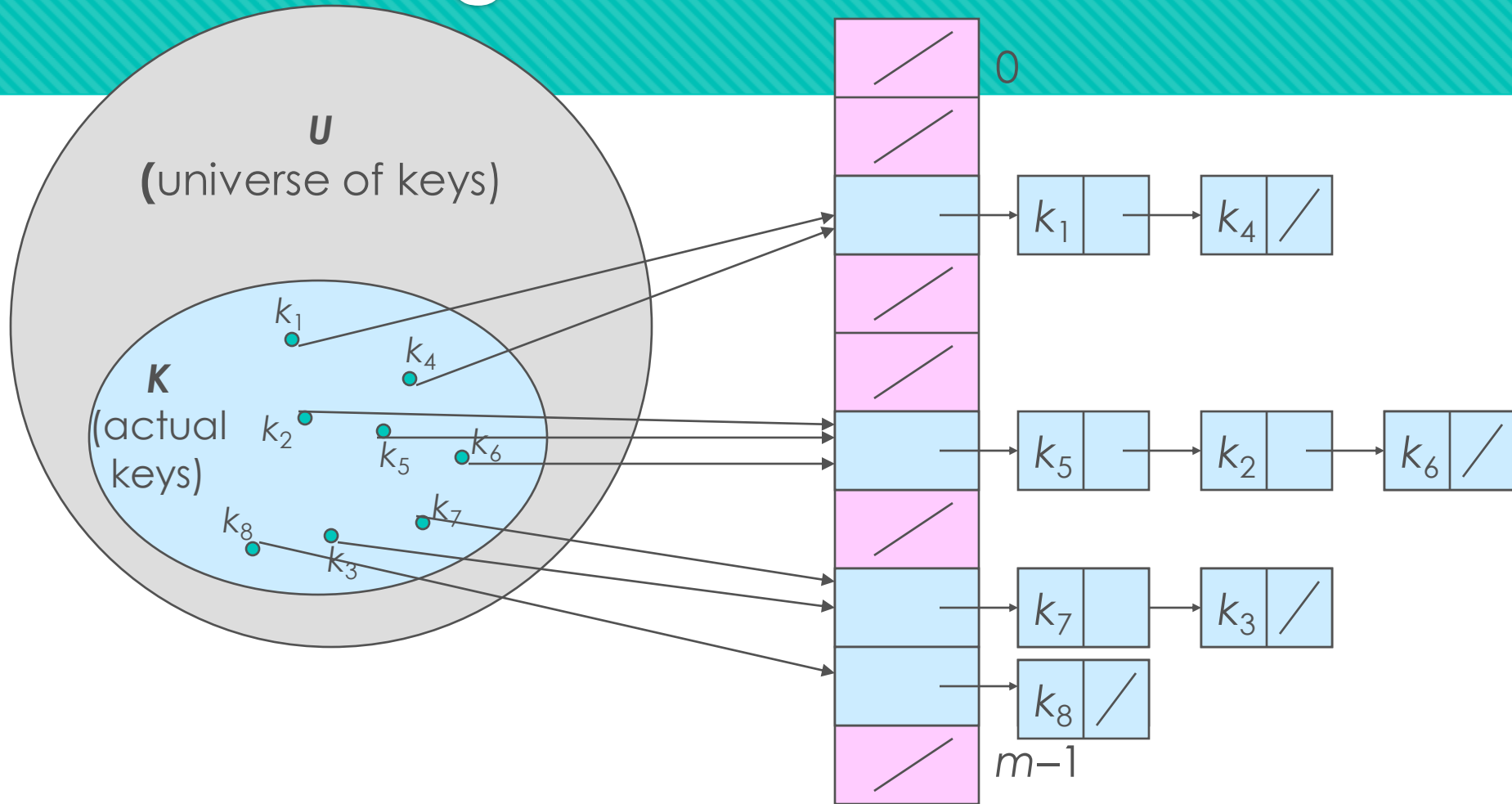
- All elements stored in hash table itself.
- When collisions occur, use a systematic (consistent) procedure to store elements in free slots of the table.



Collision Resolution by Chaining



Collision Resolution by Chaining



Linear Probing

- **Linear probing** is a collision resolution method in open addressing hash tables.
When a collision occurs (i.e., two keys map to the same index), the algorithm checks the **next available** slot in a **linear** (sequential) manner.

Linear Probing Example

Index	Value
0	
1	
2	42
3	52
4	62
5-9	

Let's assume:

- Hash function: $h(\text{key}) = \text{key} \% 10$
- Hash table size: 10 (i.e., indices 0 to 9)
- Insertion order of keys: 42, 52, 62

Step-by-step insertion:

1. Insert 42

- $h(42) = 42 \% 10 = 2$
- Index 2 is empty → insert at index 2

2. Insert 52

- $h(52) = 52 \% 10 = 2$
- Index 2 is occupied → collision!
- Check index 3 → it's free → insert at index 3

3. Insert 62

- $h(62) = 62 \% 10 = 2$
- Index 2 → occupied
- Index 3 → occupied
- Index 4 → free → insert at index 4

